

Allowing Users to Enter Data

Before we build an authentication system for creating accounts, we'll first add some pages that allow users to enter their own data. We'll give users the ability to add a new topic, add a new entry, and edit their previous entries.

Currently, only a superuser can enter data through the admin site. We don't want users to interact with the admin site, so we'll use Django's form-building tools to build pages that allow users to enter data.

Adding New Topics

Let's start by allowing users to add a new topic. Adding a form-based page works in much the same way as adding the pages we've already built: we define a URL, write a view function, and write a template. The one significant difference is the addition of a new module called *forms.py*, which will contain the forms.

The Topic ModelForm

Any page that lets a user enter and submit information on a web page involves an HTML element called a *form*. When users enter information, we need to *validate* that the information provided is the right kind of data and is not malicious, such as code designed to interrupt our server. We then need to process and save valid information to the appropriate place in the database. Django automates much of this work.

The simplest way to build a form in Django is to use a `ModelForm`, which uses the information from the models we defined in Chapter 18 to build a form automatically. Write your first form in the file *forms.py*, which should be created in the same directory as *models.py*:

```
forms.py  from django import forms

          from .models import Topic

❶ class TopicForm(forms.ModelForm):
          class Meta:
❷             model = Topic
❸             fields = ['text']
❹             labels = {'text': ''}
```

We first import the `forms` module and the model we'll work with, `Topic`. We then define a class called `TopicForm`, which inherits from `forms.ModelForm` ❶.

The simplest version of a `ModelForm` consists of a nested `Meta` class telling Django which model to base the form on and which fields to include in the form. Here we specify that the form should be based on the `Topic` model ❷, and that it should only include the `text` field ❸. The empty string in the `labels` dictionary tells Django not to generate a label for the `text` field ❹.